

Array Fun (?)

Vassilis Markos

December 9, 2024

E1. Consider the following python script:

```
1 import random
2
3 def foo(n = 100, a = 0, b = 99):
4     x = []
5     for i in range(n):
6         x.append(random.randint(a, b))
7     return x
8
9 if __name__ == "__main__":
10     a = foo()
11     b = foo(200)
12     c = foo(50, 10, 20)
13     print(f"a = {a}")
14     print(f"b = {b}")
15     print(f"c = {c}")
16
```

- (a) What is the purpose of function `foo`? What does it accept as arguments and what does it return?
- (b) Can you describe what the above script prints on screen?
- (c) Run the script to make sense of the above.

E2. Consider the following python script:

```
1 import random
2 import string
3
4 def bar(n = 100, a = 0, b = 99):
5     letters = string.ascii_lowercase
6     x = dict()
7     i = 0
8     while i < n:
9         key = "".join(random.choice(letters)\
10             for _ in range(5))
11         if key in x.keys():
12             continue
13         x[key] = random.randint(a, b)
```

```

14     i += 1
15     return x
16
17 if __name__ == "__main__":
18     a = bar()
19     b = bar(200)
20     c = bar(50, 10, 20)
21     print(f"a = {a}")
22     print(f"b = {b}")
23     print(f"c = {c}")
24

```

- (a) What is the purpose of function `bar`? What does it accept as arguments and what does it return?
 - (b) What is the purpose of “`if key in x.keys()`” in `bar`?
 - (c) Can you describe what the above script prints on screen?
 - (d) Run the script to make sense of the above.
- E3.
- (a) Write a python function named `max_three` that accepts as an argument a list of integers of length at least three, and returns the three largest elements on the list. Take care to raise appropriate exceptions when the size of the received list is less than three.
For instance, with `a = [4, 2, 1, 6, 5, 2]` as an input, `max_three` should return: 4, 5, 6 (regardless of their order).
 - (b) Modify `max_three` to accept a dictionary as an input, with strings as keys and integers as values, and return the three keys with the largest values (handling ties appropriately). Name your function `max_three_dict`.
For instance, with `a = {'a':4, 'b':5, 'c':-2, 'd':7, 'e':3}` as input, it should return: 'a', 'b', 'd'.
 - (c) Modify `max_three` to accept two arguments:
 - a list of integers, and;
 - a positive integer, n .

Now, your function should return the largest n elements from the given list, handling ties and any exceptions appropriately (e.g., the list should have at least n elements, to begin with). Name your function `max_n`.
 - (d) Modify `max_n` to accept a dictionary and a positive integer, n , as arguments, in the spirit of E2(b). That is, given a dictionary with strings as keys and integers as values, `max_n` should return the keys corresponding to the n largest values in the dictionary. Name your function `max_n_dict`.

- (e) In all the above, use `foo` from E1 as well as `bar` from E2 to help you test your functions.
- E4. (a) Write a function named `max_triplet` which accepts a list of positive integers as an argument and returns the positions of the **three consecutive numbers** in the list which correspond to the largest **three digit decimal number** (read left to right). For instance, for input `a = [4, 2, 8, 1, 9, 0, 3, 2]`, the desired output should be: 5, 6, 7, since this corresponds to 903, which is the largest three digit number in this list.
- (b) Write a function named `max_triplet_sum` which accepts a list of positive integers as an argument and returns the positions of the **three consecutive numbers** in the list with the **largest sum**. For instance, for input `a = [4, 2, 8, 1, 9, 0, 3, 2]`, the desired output should be: 2, 3, 4, since this corresponds to $8 + 1 + 9 = 18$, which is the largest sum of three consecutive integers in this list.
- (c) Write a function named `max_triplet_prod` which accepts a list of positive integers as an argument and returns the positions of the **three consecutive numbers** in the list with the **largest product**. For instance, for input `a = [4, 5, 8, 1, 9, 0, 3, 2]`, the desired output should be: 0, 1, 2, since this corresponds to $4 \times 5 \times 8 = 160$, which is the largest product of three consecutive integers in this list.
- (d) Modify the above functions to work for n -tuples instead of triplets (n should be provided as an argument). As always, take care to raise exceptions in faulty cases (e.g., when the list does not have an appropriate size). Name your functions `max_ntuple`, `max_ntuple_sum`, and `max_ntuple_prod`, respectively.
- (e) Use `foo` from E1 to help you test your functions.
- E5. This is a self-study exercise, so you are expected to heavily look up things online / on other resources. Consider the following python script:

```

1 import random
2 import math
3
4 def foobar(m = 20, n = 20, a = 0, b = 99):
5     x = []
6     for i in range(m):
7         y = []
8         for j in range(n):
9             y.append(random.randint(a, b))
10        x.append(y)
11    return x

```

```

12
13 def foobaz(x):
14     digits = lambda n: math.floor(math.log10(max(n,1)))+1
15     m = 0
16     for i in range(len(x)):
17         for j in range(len(x[i])):
18             d = digits(x[i][j])
19             if d > m:
20                 m = d
21     pad = lambda a: " "*(m - digits(a)) + str(a)+" "
22     output = ""
23     for i in range(len(x)):
24         for j in range(len(x[i])):
25             output += pad(x[i][j])
26         output += "\n"
27     return output
28
29 if __name__ == "__main__":
30     a = foobar()
31     b = foobar(15, 15)
32     c = foobar(10, 10, 10, 200)
33     print(f"a:\n{foobaz(a)}")
34     print(f"b:\n{foobaz(b)}")
35     print(f"c:\n{foobaz(c)}")
36

```

- (a) What is the purpose of foobar?
- (b) What is the purpose of foobaz?
- (c) What does lambda mean in python?
- (d) What are digits and pad computing in foobaz?
- (e) What will this print?

E6. Consider the following grid of numbers:

1	5	6	18	5
11	15	0	5	5
13	6	91	15	49
7	1	13	8	78
0	19	22	34	84

We shall refer to 2×2 blocks, such as the reddish one, merely as “blocks”, while we shall call the violetish cross-like constructs “crosses”.

- (a) Write a function named `max_block` which finds the **block with the maximum sum** in a $m \times n$ grid as the above. You can

use `foobar` from above to generate such grids and `foobaz` to print them. `max_block` should return the four positions of the maximum block as a tuple of four 2-tuples.

- (b) Write a function named `max_cross` which finds the **cross with the maximum sum** in a $m \times n$ grid as the above. You can use `foobar` from above to generate such grids and `foobaz` to print them. Your function, `max_cross`, should return the five positions of the maximum block as a tuple of five 2-tuples.
- (c) Think of a construct like blocks and crosses (tetris experience may come in handy here). Describe it in detail and then develop a python function that detects the maximum sum such construct in a $m \times n$ grid, as `max_block` and `max_cross` do above. Test your function using `foobar` and `foobaz`.